

Sequential Construct 2

Problem 1

```
.data
prompt: .asciiz "Enter your first name: "
hi_msg: .asciiz "Hi "
tail_msg: .asciiz ", MIPS assembly programming is very exciting to learn\n"
name_buf: .space 20      # holds up to 19 chars + null terminator

.text
main:
    # prompt the user
    li $v0, 4
    la $a0, prompt
    syscall

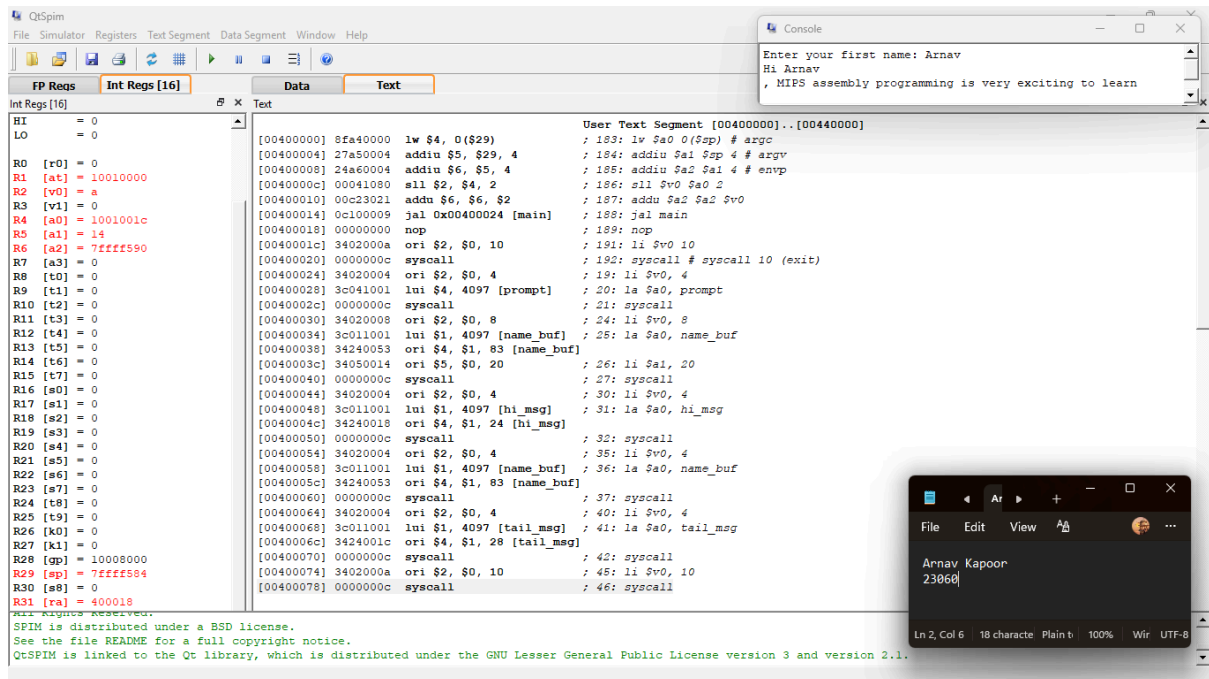
    # read the name (read_string, service 8)
    li $v0, 8
    la $a0, name_buf
    li $a1, 20
    syscall

    # print "Hi "
    li $v0, 4
    la $a0, hi_msg
    syscall

    # print the name that was typed in
    li $v0, 4
    la $a0, name_buf
    syscall

    # print the rest of the sentence
    li $v0, 4
    la $a0, tail_msg
    syscall

    li $v0, 10
    syscall
```



Problem 2

```
.data
p_prompt: .asciiz "Enter principal amount: "
r_prompt: .asciiz "Enter rate of interest: "
t_prompt: .asciiz "Enter time period: "
raw_msg:  .asciiz "\nSimple Interest (raw)    = "
abs_msg:  .asciiz "Simple Interest (absolute) = "

.text
main:
    # read Principal
    li $v0, 4
    la $a0, p_prompt
    syscall
    li $v0, 5
    syscall
    mtc1 $v0, $f4
    cvt.s.w $f10, $f4      # $f10 = Principal (float)

    # read Rate
    li $v0, 4
    la $a0, r_prompt
    syscall
    li $v0, 5
    syscall
    mtc1 $v0, $f4
    cvt.s.w $f11, $f4      # $f11 = Rate (float)

    # read Time
    li $v0, 4
    la $a0, t_prompt
    syscall
    li $v0, 5
    syscall
    mtc1 $v0, $f4
    cvt.s.w $f13, $f4      # $f13 = Time (float) (avoid $f12, used by print_float)

    # SI = (P * R * T) / 100
    mul.s $f14, $f10, $f11  # f14 = P * R
    mul.s $f14, $f14, $f13  # f14 = P * R * T
    li.s $f15, 100.0        # f15 = 100.0
    div.s $f16, $f14, $f15  # f16 = raw Simple Interest

    # print raw result
    li $v0, 4
    la $a0, raw_msg
    syscall
```

mov.s \$f12, \$f16

li \$v0, 2

syscall

print absolute result

abs.s \$f17, \$f16

li \$v0, 4

la \$a0, abs_msg

syscall

mov.s \$f12, \$f17

li \$v0, 2

syscall

li \$v0, 10

syscall

```
File Simulator Registers Text Segment Data Segment Window Help
FP Regs Int Regs [16] Data Text
Int Regs [16]
HI = 0
LO = 0
R0 [r0] = 0
R1 [at] = 10010000
R2 [v0] = a
R3 [v1] = 0
R4 [a0] = 10010065
R5 [a1] = 7ffff588
R6 [a2] = 7ffff590
R7 [a3] = 0
R8 [t0] = 0
R9 [t1] = 0
R10 [t2] = 0
R11 [t3] = 0
R12 [t4] = 0
R13 [t5] = 0
R14 [t6] = 0
R15 [t7] = 0
R16 [s0] = 0
R17 [s1] = 0
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0
R21 [s5] = 0
R22 [s6] = 0
R23 [s7] = 0
R24 [t8] = 0
R25 [t9] = 0
R26 [x0] = 0
R27 [x1] = 0
R28 [sp] = 10008000
R29 [sp] = 7ffff584
R30 [s8] = 0
R31 [ra] = 400018
See the file README for a full copyright notice.
QtSPIM is linked to the Qt library, which is distributed under the GNU Lesser General Public License version 3 and version 2.1.

[0040004c] 0000000c syscall ; 48: syscall
[00400050] 34020005 ori $2, $0, 5 ; 49: li $v0, 5
[00400054] 0000000c syscall ; 50: syscall
[00400058] 44822000 mtcl $2, $f4 ; 51: mtcl $v0, $f4
[0040005c] 468022e0 cvt.s.w $f11, $f4 ; 52: cvt.s.w $f11, $f4 # $f11 = Rate (float)
[00400060] 34020004 ori $2, $0, 4 ; 55: li $v0, 4
[00400064] 3c011001 lui $1, 4097 [t_prompt] ; 56: la $a0, t_prompt
[00400068] 34240032 ori $4, $1, 50 [t_prompt]
[0040006c] 0000000c syscall ; 57: syscall
[00400070] 34020005 ori $2, $0, 5 ; 58: li $v0, 5
[00400074] 0000000c syscall ; 59: syscall
[00400078] 44822000 mtcl $2, $f4 ; 60: mtcl $v0, $f4
[0040007c] 46802360 cvt.s.w $f13, $f4 ; 61: cvt.s.w $f13, $f4 # $f13 = Time (float) (avoid $f12, used by print_float)
[00400080] 460b5382 mul.s $f14, $f10, $f11 ; 64: mul.s $f14, $f10, $f11 # f14 = P * R
[00400084] 460d7382 mul.s $f14, $f14, $f13 ; 65: mul.s $f14, $f14, $f13 # f14 = P * R * T
[00400088] 3c0142c8 lui $1, 17096 ; 66: li.s $f15, 100.0 # f15 = 100.0
[0040008c] 44817800 mtcl $1, $f15
[00400090] 460f7403 div.s $f16, $f14, $f15 ; 67: div.s $f16, $f14, $f15 # f16 = raw Simple Interest
[00400094] 34020004 ori $2, $0, 4 ; 70: li $v0, 4
[00400098] 3c011001 lui $1, 4097 [raw_msg] ; 71: la $a0, raw_msg
[0040009c] 34240046 ori $4, $1, 70 [raw_msg]
[004000a0] 0000000c syscall ; 72: syscall
[004000a4] 46008306 mov.s $f12, $f16 ; 73: mov.s $f12, $f16
[004000a8] 34020002 ori $2, $0, 2 ; 74: li $v0, 2
[004000ac] 0000000c syscall ; 75: syscall
[004000b0] 46008445 abs.s $f17, $f16 ; 76: abs.s $f17, $f16
[004000b4] 34020004 ori $2, $0, 4 ; 79: li $v0, 4
[004000b8] 3c011001 lui $1, 4097 [abs_msg] ; 80: la $a0, abs_msg
[004000bc] 34240065 ori $4, $1, 101 [abs_msg]
[004000c0] 0000000c syscall ; 81: syscall
[004000c4] 46008b06 mov.s $f12, $f17 ; 82: mov.s $f12, $f17
[004000c8] 34020002 ori $2, $0, 2 ; 83: li $v0, 2
[004000cc] 0000000c syscall ; 84: syscall
```

Problem 3

.data

prompt: .ascii "Enter a 3-character string: "

buf: .space 5 # 3 chars + newline + null terminator

out_msg: .ascii "\nThe second character is: "

.text

main:

prompt

li \$v0, 4

la \$a0, prompt

syscall

read the 3-character string (read_string, service 8)

li \$v0, 8

la \$a0, buf

li \$a1, 5

syscall

load the second character (offset 1) using lbu

la \$t1, buf

lbu \$t0, 1(\$t1)

print label

li \$v0, 4

la \$a0, out_msg

syscall

print the character

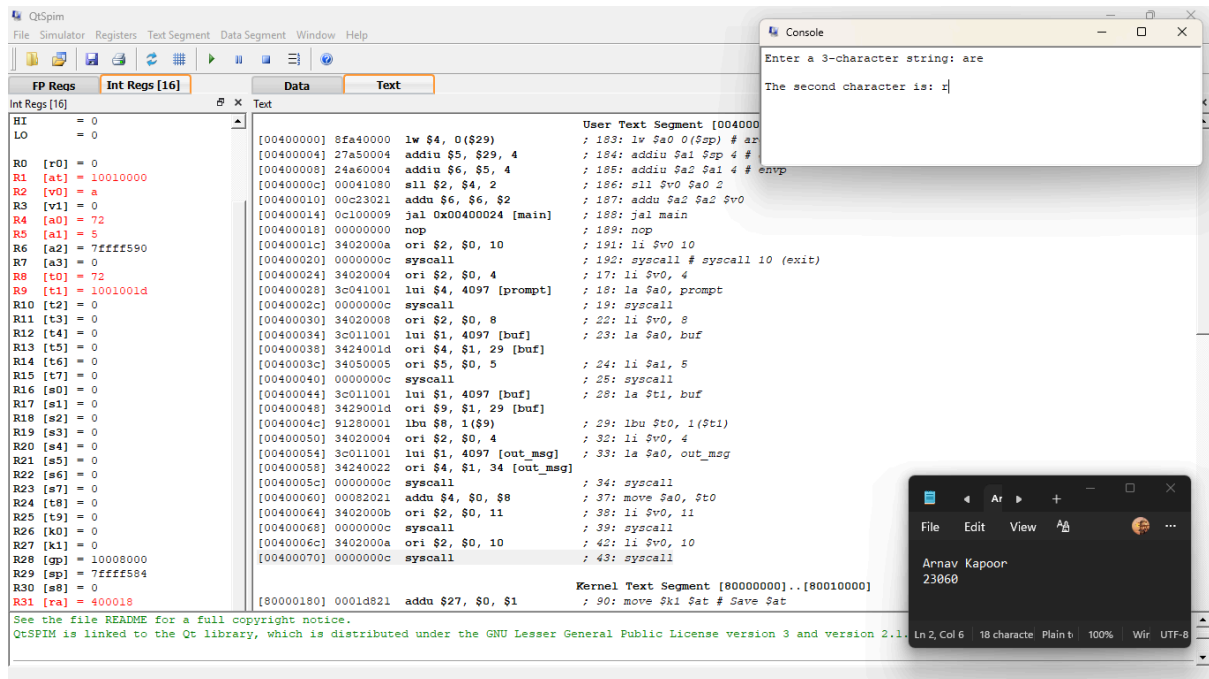
move \$a0, \$t0

li \$v0, 11

syscall

li \$v0, 10

syscall



Problem 4

.data

```
prompt_a: .asciiz "Enter first 2-character string: "  
prompt_b: .asciiz "Enter second 2-character string: "  
buf1:     .space 4      # 2 chars + newline + null terminator  
buf2:     .space 4  
result_msg: .asciiz "\nHamming comparison (per character position): "
```

.text

main:

```
# read first string  
li $v0, 4  
la $a0, prompt_a  
syscall  
li $v0, 8  
la $a0, buf1  
li $a1, 4  
syscall  
  
# read second string  
li $v0, 4  
la $a0, prompt_b  
syscall  
li $v0, 8  
la $a0, buf2  
li $a1, 4  
syscall  
  
# load base addresses  
la $t0, buf1  
la $t1, buf2  
  
# load the two characters at position 0 and position 1  
lbu $t2, 0($t0) # string1[0]  
lbu $t3, 0($t1) # string2[0]  
lbu $t4, 1($t0) # string1[1]  
lbu $t5, 1($t1) # string2[1]  
  
# compare each position  
sne $t6, $t2, $t3 # 1 if position 0 differs, else 0  
sne $t7, $t4, $t5 # 1 if position 1 differs, else 0  
  
# print the label  
li $v0, 4  
la $a0, result_msg  
syscall
```

print the two 0/1 results back to back

move \$a0, \$t6

li \$v0, 1

syscall

move \$a0, \$t7

li \$v0, 1

syscall

li \$v0, 10

syscall

The screenshot displays the QtSPIM MIPS simulator interface. The main window is divided into several panes:
1. **Registers:** Shows the state of MIPS registers. Notable values include `R1 [at] = 10010000`, `R2 [v0] = a`, `R3 [v1] = 0`, `R4 [a0] = 1`, `R5 [a1] = 4`, `R6 [a2] = 7ffff588`, `R7 [a3] = 0`, `R8 [t0] = 10010043`, `R9 [t1] = 10010047`, `R10 [t2] = 74`, `R11 [t3] = 61`, `R12 [t4] = 6d`, `R13 [t5] = 6d`, `R14 [t6] = 1`, `R15 [t7] = 1`, `R16 [s0] = 0`, `R17 [s1] = 0`, `R18 [s2] = 0`, `R19 [s3] = 0`, `R20 [s4] = 0`, `R21 [s5] = 0`, `R22 [s6] = 0`, `R23 [s7] = 0`, `R24 [t8] = 0`, `R25 [t9] = 0`, `R26 [k0] = 0`, `R27 [k1] = 0`, `R28 [gp] = 10008000`, `R29 [sp] = 7ffff57c`, `R30 [s8] = 0`, and `R31 [ra] = 400018`.
2. **Text Pane:** Displays the assembly code being executed, including instructions like `syscall`, `ori`, `lui`, `beq`, `nop`, `addu`, and `move`.
3. **Console:** Shows the program's output, including prompts for input strings and the Hamming comparison result.
4. **File Editor:** A small window in the bottom right shows a text file with the name "Arnav Kapoor" and the number "23868".

```
Enter first 2-character string: to
Enter second 2-character string: am
Hamming comparison (per character position): 11
```

See the file README for a full copyright notice.
QtSPIM is linked to the Qt library, which is distributed under the GNU Lesser General Public License version 3 and version 2.1.